Functions: def, call and return

Why functions help, how a `def` is built, what a call does, returning versus printing, and where `None` comes from.

THE QUESTION

How do you give a piece of work a name, so you can run it again on new inputs and use its answer?

THE INTUITION

`def` stores a block of code under a name and runs none of it. A call puts the arguments into the parameters, runs the body, and is then replaced by the value that `return` hands back.

KEY TAKEAWAY

Return results rather than print them. A call that reaches no `return` gives `None`. Test each small function with a few `assert` lines before you build on it.

def: store, run nothing → call: arguments in → the body runs in a new frame → return a value → the call becomes that value

1 Why functions: reuse, a name, a test

```
len("hello") → 5    min(1, 2, -10, 5) → -10    int("42") → 42
```

copy the formula

```
a1 = 3.14 * 7 * 7
a2 = 3.14 * 8 * 8
a3 = 3.14 * 10 * 10
```

3.14 typed three times

name it once, call it many times

```
def area_of_circle(r):
    return 3.14 * r * r
a1 = area_of_circle(7)   same results:
a2 = area_of_circle(8)   153.86, 200.96,
a3 = area_of_circle(10)  314.0
```

A **function** is a named block of code. You **call** it with round brackets around its **arguments**. Python has **built-in** functions such as `len`; `def` makes your own. **Reuse**: the formula is written once, so changing 3.14 is one edit. A **name**: `area_of_circle(7)` says what it works out. A **test**: check one piece alone (panel 8).

2 A `def` has a name, parameters and a body

keyword · name · parameter

```
def area_of_circle(r):          ← a colon ends the first line
    """Return the area of a circle of radius r."""
    return 3.14 * r * r
```

4-space indent: the body · docstring: what it does · return: hand back a value

```
>>> help(area_of_circle)
Help on function area_of_circle in module __main__:
area_of_circle(r)
    Return the area of a circle of radius r.
```

A **definition** is `def`, the name, the **parameters** in brackets (placeholders for the inputs) and a colon; the indented lines are the **body**. `return` ends the call and hands back the **return value**. A string on the first body line is a **docstring**, which `help` prints. A `def` only stores the function; the body runs at each call.

3 A call turns into its return value

```
def area_of_circle(r):     def area_of_rectangle(length, breadth):

arguments go
to parameters
        area_of_circle(10) + area_of_rectangle(5, 3)
   =          314.0        +          15        each call is replaced
   =                    329.0                   by the value it returned
```

```
def percent(part, whole): return 100 * part / whole
percent(30, 120) → 25.0          percent(120, 30) → 400.0
```

A call matches **arguments** to parameters by position: first to first, second to second. The body runs and the call is replaced by the value returned, as in the lecture's sum. Swap the arguments and the answer changes. Types are not checked: `area_of_rectangle("ab", 3)` gives ababab. Default and keyword arguments: next sheet.

4 `return` hands back; `print` only shows

returns the value

```
def half(n):
    return n / 2
```

only prints it

```
def show_half(n):
    print(n / 2)
```

you run	the screen shows	value kept
`x = half(9)`	*(nothing)*	x is 4.5
`y = show_half(9)`	4.5	y is None
`print(show_half(9))`	4.5 None	
`print(show_half(9) + 1)`	4.5 then TypeError	

`print` shows text to a person. `return` gives a value to the code that made the call, to store or compute with. `show_half` has no `return`, so its call gives `None`, and `+ 1` fails where `half(9) + 1` is 5.5. The notebook's `describe_roots1` prints its answer, so a list of two of its calls is `[None, None]`. Return in functions; print at the end.

5 No `return` reached: the call gives `None`

```
1 def sign_word(x):
2     if x > 0:
3         return "positive"
4     elif x < 0:
5         return "negative"
  # nothing left: return None
```

sign_word(0): lines 2 and 4 ran (grey), both tests were False, so the call gave None

```
>>> sign_word(3)
'positive'
>>> sign_word(0)  (nothing)
>>> print(sign_word(0))
None
>>> a = print("Hello")
Hello
>>> print(a)
None
```

`None` is Python's value for "no value", of type `NoneType` (`append` returns it too, see the lists sheet). A call that ends without reaching a `return` gives `None`, as does a bare `return`. `sign_word` has no branch for 0. At `>>>` a `None` result shows nothing; `print` shows it. `print` itself returns `None`.

6 `return` in a loop ends the call at once

```
1 def is_prime(n):
2     if n < 2:
3         return False
4     i = 2
5     while i * i <= n:
6         if n % i == 0:
7             return False
8         i = i + 1
9     return True
```

call	i	i * i <= n	n % i	then
25	2	True	1	i = i + 1
	3	True	1	i = i + 1
	4	True	1	i = i + 1
	5	True	0	return False
13	2	True	1	i = i + 1
	3	True	1	i = i + 1
	4	False	-	return True

True for n below 30: 2 3 5 7 11 13 17 19 23 29

`return` ends the whole call, even mid-loop, so `is_prime` needs neither the flag nor the `break` of the break and continue sheet: the first divisor returns False (25 at `i = 5`). With no divisor the loop ends and the last line returns True. That line goes after the loop, not in an `else` inside it (panel 9).

Functions that call functions, traced and tested

Max of three from max of two, followed frame by frame and checked with assert; then mistakes and practice.

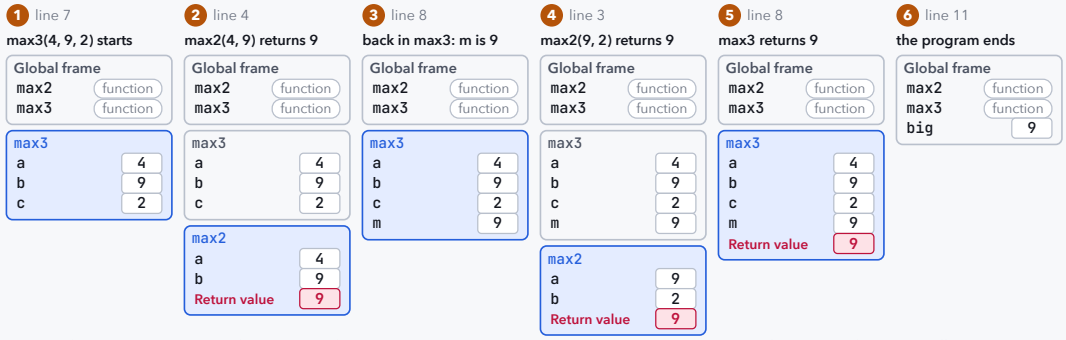
7 A function can call another; each call gets a frame, gone after its return

max3.py

```
1  def max2(a, b):
2      if a > b:
3          return a          4
4      return b              2
5
6  def max3(a, b, c):
7      m = max2(a, b)        1
8      return max2(m, c)     3 5
9
10 big = max3(4, 9, 2)
11 print(big)                6
```

numbers: the six moments on the right (of 18 call, line and return events)

output: 9



blue: the frame running now · grey frames wait · a frame appears at the call and is gone after its return · the a and b of max3 and of max2 are different names

`max3` asks `max2` for the larger of `a` and `b`, then for the larger of `m` and `c`. Each call gets a **frame** holding its parameters and the variables made in its body, its **local variables**; the program's names sit in the **Global frame**. Frames pile up as the **call stack**: `max3` waits while `max2` runs. On `return` the value goes back and the frame disappears. `return max2(max2(a, b), c)` does the same, inner call first.

8 Test a function with `assert` before you build on it

a first try at max3

```
1  def max3(a, b, c):
2      if a > b and a > c:
3          return a
4      if b > a and b > c:
5          return b
6      return c
```

four tests, run from the top

```
assert max3(1, 2, 3) == 3
assert max3(3, 2, 1) == 3
assert max3(2, 3, 1) == 3
assert max3(5, 5, 1) == 5
```

max3(5, 5, 1) gives 1: with a tie, both > tests are False

what python3 test_max3.py prints

```
Traceback (most recent call last):
  File ".../test_max3.py", line 11, in <module>
    assert max3(5, 5, 1) == 5
AssertionError
```

with max3 built from max2 (panel 7) all four pass, and it prints

```
all tests passed
```

`assert condition` does nothing when the condition is True and stops the program with an **AssertionError** when it is False. A **test** calls a function on inputs whose answer you know. Choose inputs that differ (largest first, middle, last, a tie); only the tie catches this `max3`. Test `max2` first, then what is built on it.

9 Common mistakes: what you wrote, what Python says, the fix

```
def add(a, b): print(a + b)
total = add(2, 3) + 1
```
prints 5, then `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'`
`add` gave `None`; with `return a + b` it is 6.

```
area = area_of_circle
print(area * 2)
```
`TypeError: unsupported operand type(s) for *: 'function' and 'int'`
no brackets, no call. `area_of_circle(7) * 2` is 307.72.

```
print(double(4))
def double(n): return 2 * n
```
`NameError: name 'double' is not defined`
the call ran before the `def`. Define first; it prints 8.

```
area_of_rectangle(5)
```
`TypeError: area_of_rectangle() missing 1 required positional argument: 'breadth'`
two parameters need two arguments; `(5, 3)` gives 15.

```
for i in range(2, n):
    if n % i == 0: return False
    else: return True
```
inside `is_prime`: no error, but 9 gives True
pass 1 returns either way. Put `return True` after the loop.

```
a = area_of_circle(7)
print(r)
```
`NameError: name 'r' is not defined`
`r` lived in the call's frame, now gone. `print(a)` shows 153.86.

10 Try it (answers in green)

1. `def f(x): return x * 2. print(f(3) + f(4))`?
14: each call becomes its value.

2. `def g(x): print(x * 2). y = g(5), print(y)`?
10, then None.

3. `h(n)` is only `if n > 0: return "pos"`. `h(-2)`?
None: no `return` is reached.

4. Write `is_even(n)` and test it.
`return n % 2 == 0;`
`assert not is_even(7)`

5. `first_neg(xs)` returns the first `x < 0` from a loop. `[3, -1, -5]`? `[2, 4]`?
`-1` None: no `return` for `[2, 4]`.

6. `max2(max2(1, 5), max2(7, 3))`: result, and how many calls?
7, from 3 calls to `max2`.

`def` stores, a call runs · arguments fill parameters in order · a call becomes its return value · no `return` gives `None`

Source: the functions notebook and the handwritten lecture notes on functions, nipunbatra.github.io/comp22. Outputs come from running the code with Python 3.11.11; frames, call order and loop passes from `sys.settrace`. Draw frames at pythontutor.com; which frame a name belongs to is on the scope sheet. Next: the iterables and more on functions sheet.